

MVLU COLLEGE

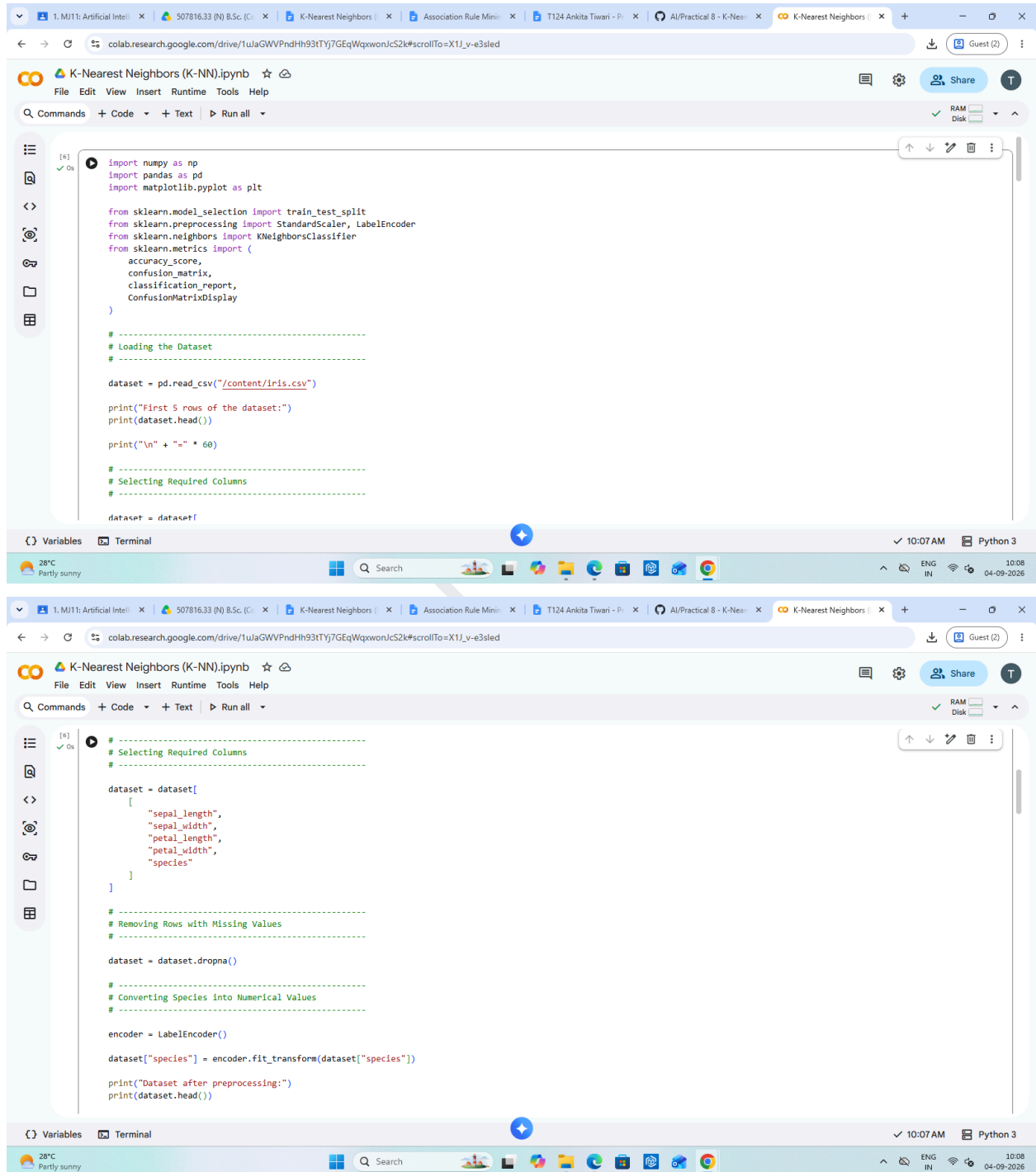
AIM:K-Nearest Neighbors (K-NN)

Implement the K-NN algorithm for classification or regression.

Apply the K-NN algorithm to a given dataset and predict the class or value for test data.

Evaluate the accuracy or error of the predictions and analyze the results.

INPUT:



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# -----
# Loading the Dataset
# -----

dataset = pd.read_csv("/content/Iris.csv")

print("First 5 rows of the dataset:")
print(dataset.head())

print("\n" + "=" * 60)

# -----
# Selecting Required Columns
# -----

dataset = dataset[
```

```
# -----
# Selecting Required Columns
# -----

dataset = dataset[
    [
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width",
        "species"
    ]
]

# -----
# Removing Rows with Missing Values
# -----

dataset = dataset.dropna()

# -----
# Converting Species into Numerical Values
# -----

encoder = LabelEncoder()
dataset["species"] = encoder.fit_transform(dataset["species"])

print("Dataset after preprocessing:")
print(dataset.head())
```

ANKITA TIWARI

TYCS

T124

MVLU COLLEGE

```
print("\n" + "=" * 60)

# -----
# Separating Input Features and Target
# -----

X = dataset.drop("species", axis=1)
y = dataset["species"]

print("Number of samples:", len(dataset))
print("\n" + "=" * 60)

# -----
# Splitting into Training and Testing Data
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\n" + "=" * 60)
# -----
```

```
# Feature Scaling
# -----

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# Creating and Training the K-NN Model
# -----

k = 5

model = KNeighborsClassifier(
    n_neighbors=k
)

model.fit(X_train, y_train)

print("K-NN Model Trained Successfully!")
print("Number of Neighbors (K):", k)

print("\n" + "=" * 60)

# -----
# Making Predictions
# -----

y_pred = model.predict(X_test)
```

ANKITA TIWARI
TYCS
T124

MVLU COLLEGE

The image displays two sequential screenshots of a Google Colab notebook titled "K-Nearest Neighbors (K-NN).ipynb". The notebook is open in a browser window with multiple tabs, including "1. MJ11: Artificial Intelligence", "507816.33 (N) 8.Sc. (C)", "K-Nearest Neighbors", "Association Rule Mining", "T124 Ankita Tiwari - P", "AI/Practical 8 - K-Nearest Neighbors", and "K-Nearest Neighbors". The notebook interface shows the "Code" tab selected, with a "Run" button and a "Share" button. The code is written in Python and is divided into two sections. The first section, labeled "Predicted Results", shows the model's predictions and the evaluation of the model's accuracy. The second section, labeled "Classification Report", shows the generation of a classification report and the visualization of the confusion matrix. The notebook is running on a Google Cloud Platform instance, with the status bar indicating "Python 3" and "10:07 AM". The bottom of the notebook shows a Windows taskbar with various application icons and a search bar.

```
[S] ✓ 0s
print("Predicted Results:")
print(y_pred)

print("\nActual Results:")
print(y_test.values)

print("\n" + "-" * 60)

# -----
# Evaluating the Model
# -----

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy * 100, "%")

print("\n" + "-" * 60)

# -----
# Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

print("\n" + "-" * 60)

# -----
# Classification Report
# -----
```

```
[S] ✓ 0s
print("\n" + "-" * 60)

# -----
# Classification Report
# -----

print("Classification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=encoder.classes_
    )
)

# -----
# Visualizing the Confusion Matrix
# -----

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=encoder.classes_
)

display.plot(cmap="Blues")

plt.title("K-NN Confusion Matrix - Iris Dataset")
plt.show()
```

ANKITA TIWARI
TYCS
T124

MVLU COLLEGE

OUTPUT:

The image displays two screenshots of a Google Colab notebook titled "K-Nearest Neighbors (K-NN).ipynb". The notebook is running a K-NN model on the Iris dataset.

First Screenshot:

```
First 5 rows of the dataset:
  sepal_length  sepal_width  petal_length  petal_width  species
0             5.1           3.5           1.4           0.2  setosa
1             4.9           3.0           1.4           0.2  setosa
2             4.7           3.2           1.3           0.2  setosa
3             4.6           3.1           1.5           0.2  setosa
4             5.0           3.6           1.4           0.2  setosa

Dataset after preprocessing:
  sepal_length  sepal_width  petal_length  petal_width  species
0             5.1           3.5           1.4           0.2      0
1             4.9           3.0           1.4           0.2      0
2             4.7           3.2           1.3           0.2      0
3             4.6           3.1           1.5           0.2      0
4             5.0           3.6           1.4           0.2      0

Number of samples: 150

Training samples: 120
Testing samples: 30

K-NN Model Trained Successfully!
Number of Neighbors (K): 5

Predicted Results:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]

Actual Results:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]
```

Second Screenshot:

```
K-NN Model Trained Successfully!
Number of Neighbors (K): 5

Predicted Results:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]

Actual Results:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]

Model Accuracy: 93.33333333333333 %

Confusion Matrix:
[[10 0 0]
 [0 10 0]
 [0 2 8]]

Classification Report:
              precision    recall  f1-score   support

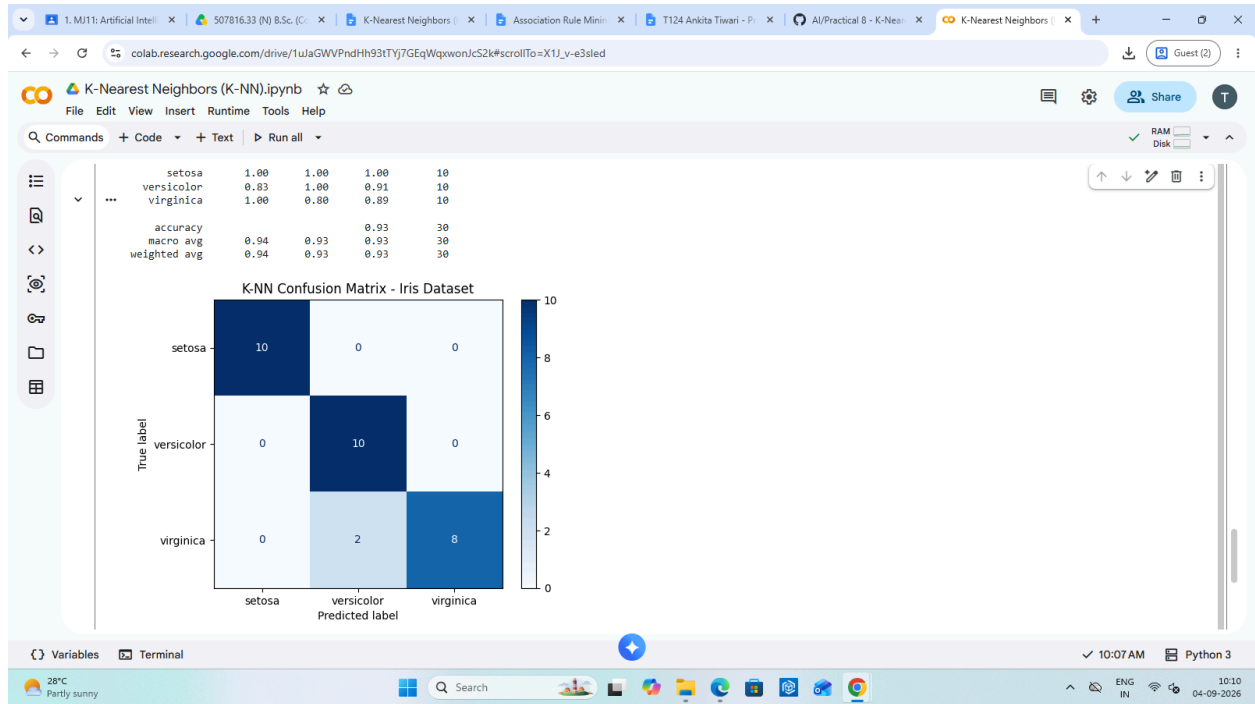
setosa         1.00        1.00        1.00        10
versicolor    0.83         0.80        0.91        10
virginica      1.00         1.00        0.89        10

accuracy         0.94         0.93         0.93        30
macro avg       0.94         0.93         0.93        30
weighted avg    0.94         0.93         0.93        30
```

K-NN Confusion Matrix - Iris Dataset

ANKITA TIWARI
TYCS
T124

MVLU COLLEGE



ANKITA TIWARI
TYCS
T124